

IA Aplicada com LLMs

Aula 07 — Medir o RAG, e a recuperação como ferramenta

Onde paramos

O pipeline responde, cita e recusa. Falta o que decide se ele serve: **estabelecer se está funcionando — e a resposta a isso não pode ser um número só.** Um número único impede o diagnóstico, e este deck trata do caso em que ele leva à correção errada.

Parte 1

Duas perguntas, duas métricas

As duas perguntas são independentes

Pergunta	Métrica	Custa chamada de modelo?
A busca trouxe o trecho correto?	<code>recall@k</code>	não
A resposta se apoia no que veio?	fidelidade	sim, uma por resposta

`recall@k` é o da Aula 06, sem mudança.

Fidelidade é novo: mede se toda afirmação da resposta pode ser derivada do contexto recuperado — ainda que a afirmação seja verdadeira no mundo.

A tabela de diagnóstico

recall@k	Fidelidade	O defeito está em	Onde mexer
baixo	qualquer	índice	corte, modelo de embedding, k
alto	baixa	prompt	contrato, ordem dos trechos, tamanho
alto	alta, e a resposta errada	corpus	documento ausente ou desatualizado

Três diagnósticos, três equipes diferentes, três correções que não se parecem.

O diagnóstico em números

```
recall@3 = 9/10 (90%) · fidelidade = 6/10 (60%)  
-> PROMPT – o trecho chega e a resposta não se apoia nele
```

O índice está adequado: em nove das dez o trecho correto foi recuperado.

Com um número só — "60% de respostas corretas" — a ação previsível seria revisar a estratégia de corte, que está correta. Dias de trabalho no lugar errado.

A terceira linha é a mais desconfortável

```
recall@3 = 10/10 · fidelidade = 10/10  
resposta: "o limite é de R$ 90,00"      <- ERRADA
```

As duas métricas **aprovam** a execução. O trecho recuperado é o Art. 4º §1º **revogado**, e a resposta é fiel a ele.

O defeito é o **modo de falha 4**, que a nota 04 dissecou, e a correção não é de recuperação: é **curadoria de corpus**.

Parte 2

Uma consulta não basta

A pergunta que uma consulta não responde

"esta despesa de R\$ 340 em Lisboa está dentro da política?"

Exige três trechos:

#	Trecho	Por quê
1	Art. 4º §1º	o teto da categoria
2	Art. 4º §2º	a exceção de viagem internacional
3	Art. 9º	quem tem alçada para aprovar

Uma única consulta com `k=3` traz os três **se e somente se** os três forem os mais similares — e não são: o §2º não menciona "R\$ 340" nem "Lisboa".

Recuperação como ferramenta

PIPELINE FIXO

busca —> responde
1 consulta, sempre
código decide

RECUPERAÇÃO COMO FERRAMENTA

agente decide QUANDO buscar
N consultas, conforme precisa
modelo decide

É a virada de arquitetura da aula: o `buscar_regulamento` vira **mais uma ferramenta** do agente da Aula 05 — e o padrão deixa de ser *prompt chaining* e passa a ser **agente**.

E o preço vem junto

Subir no espectro de autonomia cobra o que a Aula 05 estabeleceu:

- o agente pode **buscar demais** — cada consulta é contexto pago
- pode **não buscar** e responder de memória
- pode **entrar em laço** de consultas quase idênticas

```
if estado.buscas >= MAX_BUSCAS:  
    return "teto de buscas atingido; responda com o que já tem"
```

O teto de buscas é o **Orçamento** da Aula 05 aplicado a um recurso novo.

A decisão é por pergunta, não por sistema

A pergunta	A arquitetura
fato único, direto	pipeline fixo — uma busca basta
exige compor duas ou mais regras	agente — decide o que falta
tem regra determinística	nenhuma das duas — consulte a tabela

Sistemas reais têm as três. Escolher **uma** arquitetura para tudo é o erro que a Aula 05 chamou de subir no espectro sem necessidade.

O carimbo cresce

Aula	Campos
03	prompt · modelo · parâmetros
05	arquitetura
06	estratégia de <i>chunking</i> · modelo de embedding
07	k · prompt de resposta

Comparar `recall` com `k` diferente não é comparar. E trocar o prompt de resposta muda a fidelidade **sem tocar no índice**.

O que esta aula entregou

- Um pipeline que **cita** e que **recusa**, com o contrato que torna as duas coisas verificáveis
- **Duas métricas separadas**, e a tabela que transforma o par em diagnóstico
- A recuperação promovida a **ferramenta**, com o teto que a contém

E o que ela deixa

O corpus da Aula 06 tinha **dois textos verdadeiros em datas diferentes**, e o vetor não os distingue.

Aqui isso apareceu como modo de falha 4. Na **Aula 08** vira o problema central — porque memória é feita exatamente disso: fatos que envelhecem.

E há uma pendência de segurança: o documento recuperado entra no contexto com o **mesmo privilégio** da instrução do sistema. Nomeado aqui, tratado na Aula 14.